12/19 Error Handling & Building Fault tolerant Systems

Error handling is about building a fault tolerant system, so that when a system fails, it handles the failures safely. there are couple types of errors:

① Logic Errors:

(a) These happens when the program runs fine but produces the wrong results. like applying a discount twice.

(b) Unclear requirements: incorrect algorithms and unhandled edge cases that can sit in a production environment.

② Database Errors:

(a) Connection errors: App cannot reach DB because of network issues, overloaded DB, misconfigured connection pool or DB being down. these

can fail all APIs if not handled.

(b) Constraint Violation: Unique foreign keys or check constraint rejecting invalid operations (duplicate emails, invalid references)

(c) Query Errors: bad SQL joins or types that may have timeouts, deadlocks etc.

(3) External Service Errors

(a) Network issues:

Timeouts, DNS failures and partition are common when calling payments, email, storage or auth providers.

so calls must be wrapped with timeouts, retries and circuit breakers.

(b) Auth and permission errors: bad API keys, expired tokens, or missing scopes can lead to 401/403.

(c) Rate limiting: Provider throttle abusive or high-volume traffic, client should detect 429 responses and backoff using exponential backoff with jitter instead of hammering later.

(d) Service outages: even major cloud & SaaS providers go down, resilient backed systems degrade gracefully via caching fallbacks or secondary regions.

01/03/26

(e) Input Validation Errors

The boundary b/w outside world and your system is enormous: users, frontends, mobile apps, third party integrations can send something malformed. Therefore never assume data is correct. validate edge cases early.

Eg: Signup API:

Required: email, password, age

The idea is to never let bad input deep into business logic or DB where it's hard to debug.

Validation is the cheapest, strongest filter, it turns chaos at the boundary into predictable shape.

(f) Configuration errors:

The code is same binary, but behaviour changes by config, env var, feature flags, secrets, URL

Misconfig is inevitable (missing env, wrong key) and only discovered at runtime.

Eg: Required env: DATABASE_URL, PAYMENT_API_KEY, JWT_SECRET

At startup, the app runs a config validator that checks presence, basic format and environment consistency.

If anything is missing/invalid: log clearly, and fail fast.

(2) Wrong environment: wiring: Dev accidentally points staging app to production DB.

5 Graceful Error Handling

Handling errors gracefully means the system still behaves predictably safely and clearly when things go wrong.

Goals:

(a) Protect users: No raw stack traces. No something broke with no guidance, users should see clear, friendly messages.

(b) Protect system: Avoid crashes and cascades, contain failures, use fallback where safe.

(c) Help engineers: Log rich, structured details (with request IDs) so issues can be debugged and fixed quickly.

Core Patterns in a backend:

(a) Global Error Handler: A centralized middleware or filter wraps every request in a try/catch block and turns down exception into consistent HTTP responses. (JSON with

code, message? plus logs the full details.

(b) Error categories: Distinguish validation errors, auth errors (401/403), not found (404), conflict (409), rate limit and real server failures (500)

(c) User vs internal detail: Responses are short and safe (no stack traces) while log contains deeper technical info keyed by Request ID.

Global Error Handling:

It is a system where one central place in the backend catches unexpected errors, logs them, and returns a clean, consistent response to the client instead of crashing.

Pipeline

REQ → Service → Repository → DB Query
↓

The request goes to pipeline (middleware, controllers, services). If there are some unexpected errors which cannot be handled then that is transferred to the global error handler at the end of the pipeline.

The handler will:

1. Log the error with context, (route, user/tenant, if known).
2. Maps to a HTTP Request and a standardized JSON shape.
3. Hides sensitive data

Without a global handler, the system would return random HTML, stack traces, crash the system.

Error Propogation & Context.

Idea: Don't handle error when they occur, Let them bubble up, but enrich them as they move.

Repository layer: Catches low level error and wraps them.
raw DB error → DB Error (Error Fetching order 123") cause = RawFv.

Service Layer: Adds Business Context

Error Boundaries (System level safety).

They are architectural fences that stop one failure from taking over everything down.

Separate Processes/Services: If Payment crashes, Catalog still works

Timeouts: Never wait on a DB forever. timeout / handle it (retry / degrade).

Queues: Use message Queues b/w services so a spike or failure in one side doesn't immediately crash the other

Centralized Handler

(*) All requests flow through a pipeline (middleware → handler → service → repo).

(*) If any layer throws an error that isn't handled, it bubbles up to the global error handler middleware at the

end of the pipeline.

VI Security-aware error message and logs.

⇒ Errors should help engineers debug, but not help attackers learn.

⇒ User messages:

(-) Generic

(') Never show stack traces, SQL, table names, internal server error.

(') Auth flows: Avoid hints like email not found; say invalid credentials.

⇒ Logs:

(.) Detailed & Structured (JSON, requestID, userID).

(·) Do not log passwords full card numbers, raw tokens.

(·) Prefer IDs over hashes over raw PII.